

Capacity Estimation Case study & Examples

DAY 139
26 / NOV / 2025

1. E-Commerce Website

Let's you are Building an online store and need to estimate Capacity for a Black Friday Sale.

Here's how you would proceed

Define Key Metrics

Estimate 200,000 DAU and Each User makes 8 requests Per Visit, leading to a total of 1,600,000 requests.

- $\text{QPS} = 1,600,000 / 86,400 = 18.52$
- Storage Requirements : if Each User generates 5 MB of Data, the total Daily storage requirement is $200,000 \times 5 \text{ MB} = 1,000,000 \text{ MB} = 1000 \text{ GB}$
- Concurrent Users : 25% of DAU Will be active at the Same time so $200,000 \times 0.25 = 50,000$ Concurrent Users
- Conduct Load Testing : Use Apache JMeter to Simulate 50,000 Users and monitor the System's response time and Error rates
- Perform stress testing : Test the System with 250,000 Users to identify any potential bottlenecks.
- Capacity planning : Based on test results, scale the infrastructure By adding more Servers or Optimizing resources like caching
- Post Development Monitoring : once live, monitor key Metrics like DAU, QPS, and Error rates using tools like Grafana

2. Cloud Infrastructure Capacity Planning:

- **Scenario:** A company migrates its on-premises infrastructure to the cloud and needs to estimate the capacity requirements for various cloud resources.
- **Capacity Estimation:** The company analyzes historical usage data to identify resource utilization patterns and predicts future growth trends.
- **Example Metrics:** They estimate that their cloud environment requires 100 virtual machines, 10 TB of storage, and 1 Gbps of network bandwidth to support anticipated workloads.
- **Optimization Strategy:** The company implements auto-scaling policies to dynamically adjust resource allocation based on demand fluctuations, optimizing cost and performance.
- **Outcome:** By accurately estimating capacity requirements and implementing efficient resource management strategies, the company achieves cost-effective scalability and maintains high system availability in the cloud.

Challenges and Considerations

Capacity estimation in system design comes with several challenges and considerations that need to be addressed to ensure accurate predictions and optimal system performance. Here are some key challenges and considerations:

- **Dynamic Workloads:** Systems often experience fluctuating workloads due to factors like seasonal trends, marketing campaigns, or unexpected events. Predicting capacity requirements accurately in such dynamic environments can be challenging.
- **Uncertain Growth Patterns:** Forecasting future growth in terms of user base, data volume, or transaction rates is inherently uncertain. Capacity planners must account for various growth scenarios and plan for scalability accordingly.
- **Hardware Limitations:** Physical hardware constraints, such as CPU capacity, memory limits, or storage capabilities, impose limitations on system scalability. Understanding these limitations and planning for hardware upgrades or replacements is essential.
- **Software Complexity:** Modern software systems are highly complex, with numerous interconnected components and dependencies. Estimating the capacity requirements of each component and predicting their interactions accurately can be daunting.
- **User Behavior Variability:** User behavior patterns, such as peak usage times, browsing habits, or transaction volumes, can vary significantly over time. Capacity planners must analyze historical data and account for these variations in their estimations.

Best Practices for Capacity Estimation

Below are some of the best practices while doing capacity estimation:

- **Start Early:** Begin capacity estimation during the initial stages of system design to identify potential bottlenecks and scalability challenges.
- **Gather Accurate Data:** Collect and analyze accurate data on system usage, performance metrics, and workload patterns to inform capacity estimation.
- **Consider Workload Variability:** Account for variations in workload patterns, such as peak usage times and seasonal trends, when estimating capacity requirements.
- **Plan for Scalability:** Design systems with scalability in mind, utilizing techniques like horizontal and vertical scaling to accommodate future growth.
- **Regularly Review and Update:** Review capacity estimates periodically and adjust them based on changing workload patterns, technology advancements, and business requirements.

Tools and Resources for Capacity Estimation

- **LoadRunner:** LoadRunner is a performance testing tool used to simulate real-world user activity on your application. It creates virtual users to mimic actual users accessing your app. It measures how your system performs under different loads (light, normal, or heavy traffic) and identifies bottlenecks like slow responses, server crashes, or resource overuse.
- **Grafana:** Grafana is a data visualization and monitoring tool used to display real-time performance metrics of your system. Connects to data sources like databases, servers, or monitoring tools. Shows system metrics like CPU usage, memory usage, error rates, and traffic load on interactive dashboards and sends alerts when specific thresholds are crossed.
- **Load Testing Tools:** Tools like Apache JMeter, LoadRunner, and Gatling facilitate load testing to simulate real-world usage scenarios and measure system performance under various loads.
- **Monitoring Platforms:** Monitoring tools such as Prometheus, Nagios, and Datadog provide real-time insights into system performance metrics, resource utilization, and capacity trends.

Conclusion

Capacity estimation is a critical aspect of system design, ensuring that systems can handle expected workloads efficiently and reliably. By following best practices, leveraging appropriate tools and resources, and addressing challenges effectively, designers can develop robust and scalable systems that meet performance requirements and adapt to changing demands. Continuous monitoring, periodic review, and proactive planning are essential for maintaining optimal system capacity and ensuring long-term success in system design and operation.